TizenRT OS Based Data Compression Algorithms for IoT Devices

Tanmoy Roy Chamak
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, Tamil Nadu, India

Sachi Kaushik
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, Tamil Nadu, India

Shiven Singhal
School of Computer Science Engineering and Information Systems
Vellore Institute of Technology
Vellore, Tamil Nadu, India

Aditya Dadheech
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, Tamil Nadu, India

Prof. A. Anitha*
School of Computer Science Engineering and Information Systems
Vellore Institute of Technology
Vellore, Tamil Nadu, India
aanitha@vit.ac.in

Prof. S. Jenicka*
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, Tamil Nadu, India
jenicka.s@vit.ac.in

Vidisha Thapa
Samsung Research Institute
Bangalore, Karnataka, India
thapa.v@samsung.com

*Corresponding Author

2023 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS) | 979-8-3503-0611-8/23/$31.00 ©2023 IEEE | DOI: 10.1109/ICCCIS60361.2023.10425386

***Abstract*—The paper investigates the integration and adaptation of the LZ4 compression algorithm within the context of TizenRT, in addition to existing algorithms Miniz and LZMA. TizenRT, a real-time operating system tailored for Internet of Things (IoT) devices, presents unique challenges in balancing compression efficiency and resource constraints. The study evaluates the performance of LZ4 alongside Miniz and LZMA in terms of compression ratios, processing overhead, and memory utilization. Experimental results demonstrate the feasibility of LZ4's adoption in TizenRT, highlighting its potential to enhance data compression and decompression operations while adhering to the stringent requirements of IoT devices. The findings contribute to the optimization of data management strategies within the TizenRT ecosystem.**

Keywords— TizenRT, RTOS, IoT, Miniz, LZMA, LZ4, Lossless Compression.

I. Introduction

Real-time operating systems (RTOS) and IoT integration have transformed numerous sectors by providing effective and quick applications. IoT devices with RTOS support are essential for industrial automation because they enable real-time machine control and data collection from sensors. This enables greater productivity, decreased downtime, and optimized operations. IoT RTOS is used by smart grids to efficiently control power distribution systems. The RTOS can process real-time data from smart meters and sensors to balance the load, find problems, and improve energy efficiency [1]. RTOS with IoT enables intelligent transportation systems to use real-time data from traffic sensors, security cameras, and vehicle-to-infrastructure communication. This enables traffic management agencies to improve overall transportation efficiency, optimize traffic flow, and lessen congestion [2]. IoT devices with RTOS support in the healthcare industry continuously track patient vital signs and send the information for remote monitoring. This enhances patient care, enables prompt intervention, and makes it easier to identify urgent situations early [3]. In addition, RTOS is used in IoT systems for smart buildings and houses as well as for environmental monitoring, agriculture, and farming. Automation, optimization, and effective resource management are made possible by the real-time capabilities of RTOS, which guarantee prompt data processing and control. As a result of the integration of IoT and RTOS, businesses now have the ability to monitor, control, and make decisions in real-time, improving operational effectiveness, resource management, and service quality across a range of industries. RTOS-enabled IoT devices enable real-time monitoring and control of processes, resulting in enhanced productivity and reduced downtime. Intelligent transportation systems benefit from the integration of IoT and RTOS, leveraging real-time data to optimize traffic flow and improve overall efficiency. Environmental monitoring applications rely on RTOS in IoT systems to analyses real-time data from sensors and trigger appropriate actions. Agriculture and farming utilize IoT devices with RTOS to monitor soil conditions, optimize irrigation, and improve crop yields.

A. Data Compression

Data compression is essential in many fields, from communication and storage systems to scientific research and multimedia applications. The optimal use of resources is crucial given the growing amount of data being generated and sent. Techniques for data compression make it possible to

ISBN: 979-8-3503-0611-8/23/$31.00 ©2023 IEEE

reduce the quantity of data while keeping crucial information intact, which has a positive impact on processing speed, storage capacity, and transmission bandwidth. The inherent redundancy present in the majority of data is the primary driving force behind data compression. Compression methods seek to produce compact representations without losing essential data properties by locating and removing superfluous information. There are numerous data compression techniques to consider and the most effective algorithm for a particular use-case can be decided by taking into account factors such as the compression ratio and compressed file size [4]. Compression is significant in the context of big scientific data handling large-scale datasets effectively. The amount of data produced by scientific experiments and simulations is increasing exponentially, making conventional storage and processing methods insufficient. Researchers may store and communicate large datasets using efficient compression algorithms, which speeds up analysis, sharing, and cooperation [5]. The development of cloud storage platforms has increased the demand for efficient data compression. Efficient compression techniques for cloud storage systems have the advantages of reducing storage costs and optimizing data transfer in distributed environments [6]. Amid a rising need for cost-effective scalability, compression is pivotal in optimizing resource utilization. It plays an integral role in modern data management, effectively tackling issues associated with vast datasets, resource limitations, and constrained bandwidth. Compression carries significance across diverse domains, delivering advantages such as data minimization, improved storage, accelerated transfers, and efficient resource usage.

B. Lightweight Compression

A lightweight compression algorithm should concentrate on achieving a balance between compression efficiency and computational efficiency. A light-weight compression algorithm needs to have some characteristics:

Compression Ratio: The main goal of any compression algorithm is to minimize the size of the data being compressed. In order to efficiently reduce the storage or bandwidth requirements for the compressed data, a lightweight algorithm should aim for a high compression ratio.

Low Memory Footprint: Compression tables, dictionaries, and other data structures used in compression and decompression should take up the least amount of memory possible. This is crucial for IoT devices with constrained RAM resources.

Low Computational Overhead: A lightweight algorithm should be computationally effective because resource-constrained devices frequently have restricted computing capacity. It should only use a small amount of CPU time and memory when compressing and decompressing data. This minimizes the algorithm's negative effects on system performance and enables it to operate effectively on devices with limited resources.

Fast Compression and Decompression Speeds: The compression algorithm should function effectively under time constraints because devices frequently have real-time or near real-time needs. Rapid data compression and decompression speeds are critical to ensuring that IoT devices can meet their performance requirements without delays.

Energy Efficiency: Energy efficiency is essential in IoT applications to increase device battery life. Energy usage during compression and decompression operations should be kept to a minimum via the algorithm. This can be achieved by reducing computational complexity, optimizing memory access patterns, or incorporating power-saving techniques.

Minimal Dependencies: The algorithm should only depend on the bare minimum amount of complicated infrastructure or external libraries to maintain its lightweight nature. It ought to be self-contained, enabling simple integration into an environment without adding further complexity or overhead.

C. Motivation behind the Proposed Work

Our work is driven by the convergence of Real-Time Operating Systems (RTOS) and the Internet of Things (IoT), which is reshaping industries such as industrial automation, healthcare, and transportation. The fusion of RTOS and IoT empowers businesses with real-time data processing, leading to increased efficiency and higher service quality. Specifically, we focus on integrating the LZ4 algorithm into TizenRT, an RTOS optimized for resource-constrained IoT devices. LZ4 offers significant advantages in terms of speed and compression ratios, making it ideal for IoT applications where rapid data transfer and minimal memory usage are vital. Through a comparative analysis involving LZ4, LZMA, and Miniz, we aim to advance the field of IoT data compression, fostering innovation. Our motivation is to pave the way for a future marked by seamless IoT connectivity and transformative technology, facilitated by efficient algorithms like LZ4 within the TizenRT framework, ultimately revolutionizing IoT data management and enhancing the capabilities of resource-constrained devices.

D. Organization of the Paper

Section-II of this paper surveys the related work and the research papers related to data compression, IoT-based data compression and RTOS related data compression. Section-III discusses the background fundamentals of TizenRT, LZMA, LZ4 and Miniz and the key features and characteristics such as compression ratio, compression speed etc. Section-IV describes the proposed work which includes the installation of TizenRT OS and the adoption of LZ4 in TizenRT OS. Section-V covers the experimental analysis performed in the TizenRT OS and discusses the metrics such as compression ratio calculated using LZ4. Section-VI states the conclusion with future scopes.

II. Related Work

This section discusses the related work with respect to data compression techniques, IoT- based data compression and data compression with various RTOS.

A. Data Compression Techniques

Saniie et al. [7] investigate the development of learning models for massive data compression by putting several machine learning techniques into practice. Its foundations are the quick chirplet transform technique and wavelet packet transformation. A study of current data compression methods, categorized according to elements including data quality and coding schemes, is provided by Jayasankar et al. [8]. An extensive summary of the applications of data compression methods in cloud computing, multimedia, and sensor networks is given. It is necessary to take into account the needs of various data kinds while choosing compression strategies. An effective and secure data compression method using adaptive Huffman coding and chaos theory is presented in [9]. Wang et al.'s [10] lossy compression technique for encrypted binary images makes advantage of the Markov random field (MRF)

model. Concatenated down sampling and LDPC-based encoding are used to create the compressed image, and several down sampling algorithms are used to enhance the quality of the reconstructed image.

B. IOT with Data Compression

Correa et al. [11] demonstrated that the Internet of Things (IoT) is a good platform for industrial applications and gives a thorough assessment of the literature on lossy data compression algorithms and feature a high compression ratio that minimizes compression errors while maintaining the integrity of the data. An adaptive data compression scheme (ADCS) [12] has proposed to effectively regulate the compression rate and energy consumption of IoT devices in the cloud-based IoT network. A mathematical programming model called mixed-integer linear programming (MILP) is utilized as an optimization technique to produce a scheme to reduce total traffic power of cloud-based IoT network. Ketshabetswe et al. [13] suggested an improved data compression algorithm that made use of Huffman coding tables to encode residue samples. The data was significantly compressed in the simulation which lowered the bitstream and increased energy efficiency. Deepu et al. [14] introduced the hybrid data compression scheme for the reduction of power consumption in IoT wireless sensors. In this approach, both lossy and lossless compression techniques were combined to enable adaptive data rate selection and power savings during wireless transmission. Notable compression ratios of 2.1x for lossless and 7.8x for lossy compression were achieved when applied to electrocardiogram (ECG) data, with data fidelity being preserved.

C. Data Compression on Various IoT Devices

Prasad et al. [15] emphasized that the satellite's on-board computer manages commands and subsystem health. A Field Programmable Gate Array (FPGA) uses CCSDS 1.2.3 algorithm to compress hyperspectral images before downlinking. Camtepe et al. [16] introduce a lightweight compression and encryption solution called Compcrypt based on Asymmetric Numeral Systems (ANS) providing three approaches all making use of a pseudorandom bit generator (PRBG). For applications with limited resources, it offers a trade-off between compression ratio, encryption strength, and computational complexity. Tsai and Kuo [17] introduced a context-adaptive Golomb-Rice code, an adaptive linear predictor, and a reduced complexity lossless ECG compression technique. The technique seeks to reduce the amount of time needed for transmission and storage for wearable ECG devices. Lu et al. [18] introduced a reliable data compression scheme for Sensor-Cloud Systems, combining lossy and lossless compression methods. The proposed lossy algorithm, DFan, achieved high compression ratios while retaining data accuracy. Experimental results demonstrated compression ratios of 4.21x and 3.88x for the hybrid scheme, and 6.42x and 5.1x for lossy compression with different datasets.

III. Background Fundamentals

A. TizenRT

TizenRT, an open-source operating system, is primarily developed by Samsung Electronics, a major technology company actively engaged in the Tizen platform's advancement, encompassing TizenRT for IoT devices. It constitutes part of the broader Tizen ecosystem, which encompasses Tizen OS utilized in smartphones, smart TVs, wearables, and various other devices. TizenRT, an open-source operating system, is dedicated to IoT devices. It is derived from the Tizen operating system, developed by the Linux Foundation. This lightweight and flexible platform is thoughtfully designed for IoT devices constrained by limited resources. It operates as a real-time operating system (RTOS), creating a conducive environment for diverse IoT applications.

TizenRT offers a range of remarkable features:

Lightweight and Scalable: TizenRT is a great option for Internet of Things (IoT) devices with limited resources because of its lightweight and scalable design. It works flawlessly on devices with modest memory and processing power because of its efficiency and adaptability.

Real-Time Operating System: Important events and tasks are handled promptly thanks to TizenRT's real-time operating system features. It is hence the best option for Internet of Things devices that require real-time performance.

Modular Architecture: TizenRT's modular architecture allows developers to tailor the operating system to the specific requirements of their Internet of Things devices.

Interoperability: TizenRT supports a broad range of IoT standards and communication protocols, bridging the gap between disparate IoT environments. This makes it possible to integrate different cloud platforms and IoT ecosystems with ease.

Security: Because IoT devices are naturally prone to security breaches, TizenRT has built-in security protections and processes to secure user data.

Application Framework: To create Internet of Things apps, TizenRT provides a developer-friendly environment. Because of its extensive framework's support for a wide variety of programming languages and tools, developers can more easily produce creative and potent solutions for the various demands of the IoT environment.

Open-Source: Anyone in the community is welcome to contribute to TizenRT, as it is an open-source project. IoT developers are encouraged to collaborate and be innovative by this open approach, which advances IoT technology.

TizenRT's primary goal is to provide a dependable and adaptable platform for creating IoT solutions, simplifying the deployment and management of IoT devices in various industries, including home automation, smart appliances, industrial automation, and healthcare, among others. The common features in various compression algorithms studied in the paper are specified in table I.

B. LZMA

The Lempel-Ziv-Markov chain algorithm (LZMA) is a data compression approach that combines the LZ77 algorithm with the arithmetic coding scheme's context modelling strategies. Igor Pavlov created it in 1998, and it is now widely used in many programs and file formats.

Additional features:

Dictionary-based Compression: To identify recurring patterns in the input data, LZMA makes use of a sliding dictionary. LZMA accomplishes effective compression by substituting references for these patterns. The dictionary's size can be changed to affect compression effectiveness; larger dictionaries provide longer matches and possibly higher compression ratios.

Context Modelling: Dictionary-based compression is combined with context modeling techniques in LZMA. It estimates the likelihood of each symbol (byte) by using a Markov chain, which is predicated on the symbols that came before it. LZMA achieves higher compression by adjusting to the specific qualities of the data.

C. Miniz

Miniz is a small data compression library that provides a straightforward and effective method for both binary data compression and decompression. Rich Geldreich created Miniz as an open-source project with the goal of having small code that is simple to integrate into a variety of applications.

Additional feature:

Lightweight and Portable: Miniz is a lightweight and portable tool that can be seamlessly integrated into projects without adding a lot of complexity or overhead. It is implemented as a single C source file.

D. LZ4

LZ4 is a well-known lightning-fast compression algorithm because of its remarkable speed and memory-saving efficiency. It was created by Yann Collet, and since its launch in 2011, LZ4 has grown in popularity for a variety of uses.

Additional feature:

Block-based Structure: The block-based technique used by LZ4 splits input data into discrete blocks and processes each one separately. This allows for the processing of multiple blocks in parallel, greatly increasing performance on contemporary multi-core CPUs. Because of its low memory constraints and real-time compression demands, LZ4's unique feature along with other beneficial qualities make it the perfect choice for compression in the TizenRT environment. In this context, compression speed and ratio are critical factors.

E. Comparison between LZMA, Miniz, LZ4

TABLE I. COMPARISON BETWEEN LZMA, MINIZ, LZ4

Features	Algorithm		
	LZMA	**Miniz**	**LZ4**
Compression Efficiency	As file size reduction is crucial when storing and preserving data, LZMA is renowned for its higher compression performance. Although it is a complex algorithm, it can produce impressive compression, the processing performance is slowed as the result.	Miniz achieves a balance between speed and compression effectiveness. It is suitable for situations where a decent trade-off between size reduction and processing time needs to be achieved since it offers modest compression while maintaining faster compression speeds.	LZ4 prioritizes compression speed over extremely high compression ratios. It offers respectable compression efficiency but focuses on lightning-fast processing, which makes it ideal for scenarios where speed is paramount and a slightly smaller reduction in file size is acceptable.
Compression Ratio	With its exceptionally high compression ratios, LZMA stands out because it can drastically reduce the quantity of input data. As a result, it is especially helpful in situations when storage space is at a premium and compression ratios are more important than processing speed.	Miniz offers adequate compression ratios that result in acceptable file size reduction and expedite compression. It is ideal for applications where a moderate size reduction is allowed and speedier compression is sought due to this balance.	LZ4 offers lower compression ratios compared to LZMA and even Miniz. However, its major advantage lies in its ability to compress and decompress data at unparalleled speeds. It is ideal for use cases where the main goal is to achieve rapid data processing with minimal impact on compression ratios.
Compression Speed	LZMA is purposefully made to compress at slower rates in order to maximise compression effectiveness. It performs well in situations when maximising compression ratios is the primary goal, even if it results in longer processing times.	Miniz is optimized for faster compression speeds compared to LZMA. It strikes a balance between compression efficiency and speed, making it suitable for applications where timely data compression is important, and a reasonable reduction in file size is acceptable.	LZ4 has been known for having an amazing compression rate. One of the fastest compression algorithms on the market, it is a great option when a slightly lower compression ratio is acceptable and speed is a key consideration. High-speed applications often employ it for real-time data processing.

IV. PROPOSED WORK

This subsection explores the installation of TizenRT on a Linux based OS (Ubuntu) for IoT development purposes and the integration of LZ4 Compression in TizenRT.

A. Installation and Environment Analysis

(i) Installation of Development Environment: Docker was installed to streamline the management of necessary libraries and toolchains, eliminating the need for manual handling. Within the Docker container, all essential components for TizenRT development are bundled together. The TizenRT source code was obtained by cloning it from the official GitHub repository.

(ii) Building TizenRT Using the Interactive Tool: TizenRT provides an interactive tool (`dbuild.sh`) where one is prompted to select an option among multiple choices. Navigating to '`TizenRT/os/`' directory '`./dbuild.sh menu`' command was used. The process started with choosing the '`rtl8721csm`' board from a list of a supported boards for this use case. Then, '`loadable_apps`' was selected as the board configuration from a list of supported configuration available for that particular board. Finally, the '`Build with Current Configuration`' option was chosen to test basic functionality.

(iii) Configurations for Compression: We utilized the interactive tool (`dbuild.sh`) to select '`Modify Current`

Configuration' and specifically chose the Compression option to investigate compression configurations. Initially, we conducted a test using the default (Miniz) compression type with a block size of 16384 (bytes).

B. Integration of LZ4 in TizenRT OS

The integration of LZ4 into TizenRT required the creation of a new directory structure within 'TizenRT/external/' to house essential files, specifically 'lz4.c' and 'lz4.h', sourced from the official LZ4 repository. This integration was further streamlined by making necessary configuration adjustments. In the 'TizenRT/os/compression/' directory within 'Kconfig' file, the 'COMPRESSION_TYPE' configuration option was expanded to include support for LZ4 compression (3 = LZ4). Similarly, in the 'Makefile' file, the '-DLZ4=3' flag was added to 'CFLAGS' to enable LZ4 compression, in 'compress.c', LZ4 compression logic was integrated, involving LZ4 header inclusion, the integration of the 'LZ4_compress_default()' for block compression, and related error handling. The 'compress_read.c' file saw the addition of LZ4 decompression logic, including the utilization of the 'LZ4_decompress_safe()'. Lastly, in 'TizenRT/os/tools/compression/src/compress.c', LZ4 support was added for 'CONFIG_COMPRESSION_TYPE' set to 3.

These changes introduce support for LZ4 compression alongside existing LZMA and Miniz compression in TizenRT.

V. Experimental Data and Analysis

A. Binary Files

Binary Representation: Binary files employ a format that combines binary digits (0s and 1s) to represent data. The smallest unit of data is a bit, and groups of bits are called bytes. Bytes can represent letters, numbers, or other data kinds.

Complex Data: Binary files are capable of storing complex data structures like graphics, which need exact byte placements to represent the colors and pixels.

Non-Textual Data: While binary files can hold a variety of non-textual data kinds, including multimedia files (images, audio, and video), built software, executable programs, databases, and more, text files are generally used to store human-readable characters.

Binaries for Configuration: TizenRT sets numerous parameters and options for various components using configuration files. It is possible to compile these configuration files into binary form for quick runtime application and processing.

In this we explored the files such as app1.uncomp and app2.uncomp having an initial file size of 16.2 kb and 878.6 kb respectively and other two files such as app1 and app2 which are the compressed binaries.

B. Experiment Analysis with Compression Ratios

The success of a compression technique in lowering the size of application binaries and related resources is shown by the compression ratio, which is a crucial parameter. It measures how much data is compressed during the compression process. The Compression Ratio (CR) is computed using the following formula in eqn (1).

$$CR = \left(\frac{Size\ Reduction}{Original\ size}\right) * 100 \qquad (1)$$

Where, Size Reduction equals the size of Uncompressed Binary - Size of Compressed Binary, Original Size equals the Size of the Uncompressed Binary.

The compression ratios of LZMA, Miniz and LZ4 are depicted in Table II for the files app1 and app2 (binary files) for different block sizes. From the tables, it is clearly seen that, in terms of compression ratio, LZMA performs better than other algorithms for the binary files. The same is represented as graphs in Figure 1 and Figure 2 for the files app1 and app2 respectively.

TABLE II. Comparison of Compression Ratios

Algorithm		2KB	4KB	6KB	8KB	10KB	12KB	16KB
LZMA	app1	43.45	45.68	46.57	47.73	47.51	47.45	48.24
	app2	56.87	60.33	61.84	62.76	63.42	63.85	64.39
Miniz	app1	40.46	42.21	42.48	43.39	43.09	42.60	42.63
	app2	54.00	56.92	58.00	58.63	59.08	59.37	59.68
LZ4	app1	23.11	24.05	24.61	25.99	25.78	24.99	26.29
	app2	33.88	36.14	37.15	37.79	38.27	38.64	39.10

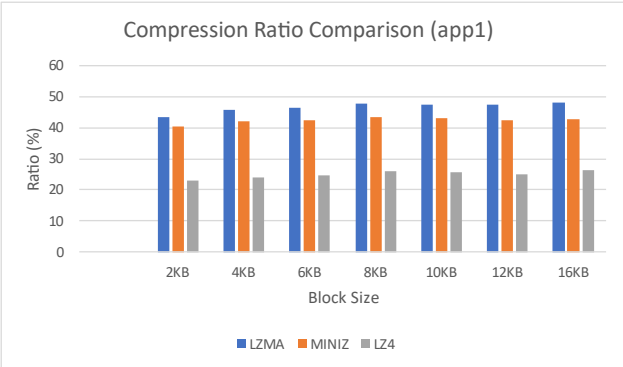

Fig. 1. Compression Ratio Comparison (app1)

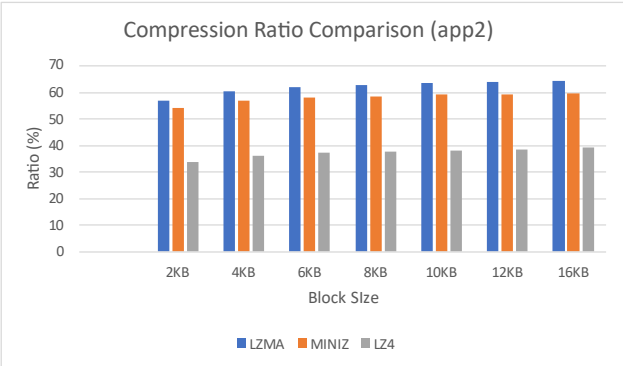

Fig. 2. Compression Ratio Comparison (app2)

C. Results Discussion with Performance Metrics

In this study, we successfully integrated the LZ4 compression algorithm into the TizenRT operating system, expanding its data compression capabilities alongside Miniz and LZMA. To evaluate the performance of these algorithms, we conducted extensive experiments across various file sizes on two representative IoT applications (app1 and app2). Our results manifested that LZ4 consistently outperformed LZMA in terms of compression speed while maintain competitive compression ratios. Miniz, on the other hand, comes with a balance between compression speed and compression ratios, making it suitable a wide range of IoT applications.

VI. Conclusion and Future Enhancement

Our study integrates the LZ4 compression algorithm in parallel to Miniz and LZMA to show the improved the capabilities of the TizenRT OS. We have repeatedly shown that LZ4 outperforms LZMA in terms of compression speed while maintaining similar compression ratios through testing with different block sizes and IoT applications. Miniz offers a stable performance that is appropriate for a variety of Internet of Things applications at the same time.

Our results emphasize how crucial it is to optimize data compression techniques for Internet of Things devices with limited resources. LZ4's integration into the TizenRT ecosystem broadens the options for compression while also enhancing the system's capacity to handle real-time data processing. This work is a major step toward increasing IoT device efficiency, lowering energy consumption, and enhancing overall performance.

Our integration of LZ4, Miniz, and LZMA for IoT data compression has shown great promise within the TizenRT framework. In particular, performance improvement via optimized algorithms will be the top priority in our future work, along with machine learning techniques and adaptability to changing data patterns. Through these efforts, TizenRT's data compression capabilities will be improved and made more in line with the changing needs of Internet of Things applications.

VII. References

[1] Amin, S. M., & Wollenberg, B. F. (2005). Toward a smart grid: power delivery for the 21st century. IEEE power and energy magazine, 3(5), 34-41.

[2] De Oliveira, L. F. P., Manera, L. T., & Da Luz, P. D. G. (2020). Development of a smart traffic light control system with real-time monitoring. IEEE Internet of Things Journal, 8(5), 3384-3393.

[3] Rockwood, M., Joshi, V., Sullivan, K., & Goubran, R. (2014, June). Using a real-time operating system for multitasking in remote patient monitoring. In 2014 IEEE International Symposium on Medical Measurements and Applications (MeMeA) (pp. 1-5). IEEE.

[4] Al-Laham, M., & El Emary, I. M. (2007). Comparative study between various algorithms of data compression techniques. IJCSNS, 7(4), 281.

[5] Zou, H., Yu, Y., Tang, W., & Chen, H. M. (2014, May). Improving I/O performance with adaptive data compression for big data applications. In 2014 IEEE International Parallel & Distributed Processing Symposium Workshops (pp. 1228-1237). IEEE.

[6] Hossain, K., Rahman, M., & Roy, S. (2019). Iot data compression and optimization techniques in cloud storage: current prospects and future directions. International Journal of Cloud Applications and Computing (IJCAC), 9(2), 43-59.

[7] Saniie, J., Govindan, P., Wang, B., Zhang, X., Lu, Y., & Oruklu, E. (2022, October). Machine Learning and Modeling of Ultrasonic Signals for High-Fidelity Data Compression. In 2022 IEEE International Ultrasonics Symposium (IUS) (pp. 1-9). IEEE.

[8] Jayasankar, U., Thirumal, V., & Ponnurangam, D. (2021). A survey on data compression techniques: From the perspective of data quality, coding schemes, data type and applications. Journal of King Saud University-Computer and Information Sciences, 33(2), 119-140.

[9] Usama, M., Malluhi, Q. M., Zakaria, N., Razzak, I., & Iqbal, W. (2021). An efficient secure data compression technique based on chaos and adaptive Huffman coding. Peer-to-Peer Networking and Applications, 14, 2651-2664.

[10] Wang, C., Li, T., Ni, J., & Huang, Q. (2019). A new MRF-based lossy compression for encrypted binary images. IEEE Access, 8, 11328-11341.

[11] Correa, J. D. A., Pinto, A. S. R., & Montez, C. (2022). Lossy data compression for iot sensors: A review. Internet of Things, 19, 100516.

[12] Al-Kadhim, H. M., & Al-Raweshidy, H. S. (2021). Energy efficient data compression in cloud based IoT. IEEE Sensors Journal, 21(10), 12212-12219.

[13] Ketshabetswe, K. L., Zungeru, A. M., Mtengi, B., Lebekwe, C. K., & Prabaharan, S. R. S. (2021). Data compression algorithms for wireless sensor networks: A review and comparison. IEEE Access, 9, 136872-136891.

[14] Deepu, C. J., Heng, C. H., & Lian, Y. (2016). A hybrid data compression scheme for power reduction in wireless sensors for IoT. IEEE transactions on biomedical circuits and systems, 11(2), 245-254.

[15] Prasad, A., Jain, Y., Joshi, N., Gupta, N., Singhania, V., & Sreedharan, Y. (2020, March). Interfacing Architecture between Telemetry and On-Board Computer for a Nanosatellite. In 2020 IEEE Aerospace Conference (pp. 1-6). IEEE.

[16] Camtepe, S., Duda, J., Mahboubi, A., Morawiecki, P., Nepal, S., Pawłowski, M., & Pieprzyk, J. (2021). Compcrypt–lightweight ANS-based compression and encryption. IEEE Transactions on Information Forensics and Security, 16, 3859-3873.

[17] Tsai, T. H., & Kuo, W. T. (2018). An efficient ECG lossless compression system for embedded platforms with telemedicine applications. IEEE Access, 6, 42207-42215.

[18] Lu, S., Xia, Q., Tang, X., Zhang, X., Lu, Y., & She, J. (2021). A reliable data compression scheme in sensor-cloud systems based on edge computing. IEEE Access, 9, 49007-490